Run the 7-step process on a mini scenario — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely written your own 7 steps first. Write your answer in `practice/seven_step_practice.md` as you go.

- Hint 1 — Problem, Requirements, Architecture
- Hint 2 — Components, Data Flow, Implementation Plan, Coding Tasks

Hint 1 — Problem, Requirements, Architecture

Problem — restate it more precisely than the one-liner: who is this for, what does "flagged" actually mean to them (a label? a notification? a separate list?), and what happens if nothing is urgent today?

Requirements — split into "must do" (read new emails, judge urgency, surface the urgent ones somewhere) and "must never do" (never delete or modify the original email, never flag something as urgent with no way to see why).

Architecture — decide: is this a single agent that reads and judges in one pass, or two agents (a reader/summarizer, then a judge)? Tie this back to Doc11's real question — "one agent, or many?" — a task this small usually doesn't need multiple agents, but write down *why* you decided that, not just the decision.

Before you commit to a design: the one-sentence request has hidden ambiguity a senior engineer would flag before designing anything: "daily emails" — read once a day, or continuously? "the team's emails" — one shared inbox, or every team member's own inbox? "urgent" — is there an existing definition (keywords? sender? a deadline mentioned in the text?) or does the system have to invent one?

Write down every place this request is genuinely ambiguous, and what you'd ask before committing to a design — this is the same skill Doc16's "unclear, conflicting requirements" exercise tests, applied here at a smaller scale. A design built on an unstated assumption you never questioned is a design that's wrong the moment someone points out the assumption was false.

Hint 2 — Components, Data Flow, Implementation Plan, Coding

Tasks

Components — name them like real pieces of code, not vague ideas: an email-fetching function, an urgency-judging agent (or function), a place flagged emails get written to (a list? a file? a real inbox label?).

Data Flow — trace one email's entire path: fetched → passed to the judge → judge returns urgent/not-urgent plus a reason → if urgent, written to the flagged output. Write this as an actual sequence, not a description.

Implementation Plan — decide the build order and *why*: usually, get the fetch-and-judge pipeline working on a hardcoded test email first, before wiring up a real inbox — the same "test the logic before the plumbing" instinct as Doc01's `require_env()` pattern.

Coding Tasks — break the plan into pieces small enough to each be one sitting of work: "write the urgency-judging function," "test it against 5 example emails," "wire up the real email source," "wire up the flagged-output destination."

What could go wrong, and what's out of scope: a senior-level plan also names what could go wrong at each Data Flow step, not just the happy path: what if the email source is unreachable (a real API failure, Doc02's territory)? What if the urgency-judge genuinely can't tell (Doc07's "when NOT to use an agent" — maybe some emails should just be flagged "needs a human to decide" instead of forcing a yes/no)? What if two emails about the same topic should be flagged together, not separately?

Write your Implementation Plan with an explicit "what I'm deliberately NOT handling in v1" line — a real senior plan draws a line around scope on purpose, rather than trying to handle everything at once.

Full solution: Show me the solution